Coin Flip Cryptography (Quadratic Residues + CRT)

Slide 1 — Title

Coin Flip Cryptography
Fair remote coin-tossing using modular arithmetic, quadratic residues, and the Chinese Remainder Theorem (CRT).

- Names / Course / Date
- Demo parameters (for class demo): $p = 31$, $q = 43$, $n = 1333$

Slide 2 — Problem: Why "normal" coin toss fails online

Your slide is correct. When Alice and Bob are remote, a simple "I call heads" is not trustworthy.

- **Cheating by Alice:** can claim "heads" after seeing Bob's message.
- **Cheating by Bob:** can change his call after seeing Alice's message.
- **Faked numbers / messages:** the network allows editing/replaying messages.

Goal: a protocol where neither side can force the outcome.

Slide 3 — Key mathematical concepts (what you already have)

Your content is correct; I suggest tightening wording:

- **Prime numbers:** used to build a hard-to-solve composite number.
- **Modular arithmetic:** values wrap around mod n (like clock arithmetic).
- **Squaring + square roots mod n:** easy to square, hard to "unsquare" without extra secret information.
- **CRT:** lets Alice combine solutions mod p and mod q into solutions mod n.

Slide 4 — Protocol overview (1 slide)

Setup

1. **Alice** chooses primes p, q (secret) and sends $n = p \cdot q$ to Bob.

Commitment by Bob

2. **Bob** chooses random α and computes $A = \alpha^2 \bmod n$.
3. Bob sends A to Alice.

Alice's advantage (because she knows p and q)

4. Alice computes the **four** square roots of A modulo n.
5. Alice picks one root β and sends it to Bob.

Bob's check

6. Bob computes `gcd(α + β, n)` and `gcd(α − β, n)`.

- If either gcd is a non-trivial factor, Bob learns `p` or `q` → **Bob wins**.
- Otherwise gcd is `1` or `n` → **Bob learns nothing** → **Alice wins**.

Fairness intuition: Alice has a 1/2 chance to send `β = ±α` (safe), otherwise Bob can factor `n`.

Slide 5 — Why p ≡ q ≡ 3 (mod 4)?

This is correct to mention because it makes square roots easier.

- Choose primes where `p ≡ 3 (mod 4)` and `q ≡ 3 (mod 4)`.
- Then a square root of `A` mod `p` can be computed as:

[x_p \equiv A^{(p+1)/4} \pmod{p}]

Same for `q`.

Slide 6 — Demo: Step 1 (Alice sends n)

What to show (screenshot from notebook output):

- Notebook **Cell 2 output** (your first code cell)

Expected output:

```
Alice sends n = p*q = 1333 to Bob.
```

Talking line: "Alice only sends n; p and q stay secret."

Slide 7 — Demo: Step 2 (Bob commits using squaring)

What to show:

- Notebook **Cell 5 output**

Expected output:

```
A: 669
```

Talking line: "Squaring hides α: Bob reveals A but keeps α secret."

Slide 8 — Demo: Step 3 (Alice computes roots mod p and q)

What to show:

- Notebook **Cell 7 output**

Expected output:

```
The value of x_p congruent to +/- 7 (mod 31)
The value of x_q congruent to +/- 14 (mod 43)
```

Talking line: "Because Alice knows p and q, she can compute square roots modulo each prime."

Slide 9 — Demo: Step 4 (Alice computes 4 roots using CRT)

What to show:

- Notebook **Cell 10 output**

Expected output:

```
Four square roots of A modulo n:
  beta = 100
  beta = 1061
  beta = 272
  beta = 1233
```

Talking line: "There are 4 possible square roots modulo n=pq (for Blum primes)."

Slide 10 — Demo: Step 5 (Alice sends β; Bob checks gcd)

What to show:

- Notebook **Cell 13 output** (shows α, n, and β)
- Notebook **Cell 14 and 15 outputs** (the two gcd checks)

Expected outputs:

```
alpha = 100, n = 1333
Alice sends beta = 272
```

```
gcd(alpha + beta, n) = 31
```

```
gcd(alpha - beta, n) = 43
```

Talking line: “Since β is not ±α, the gcd reveals factors of n (31 and 43).”

Slide 11 — One clean “Result” slide (recommended)

If you want only ONE screenshot for results, use:

- Notebook **Final Summary cell output**

Expected output block:

```
=== Summary ===
p = 31, q = 43, n = 1333
Bob chose alpha = 100
A = alpha^2 mod n = 669
Roots mod p: ±7 (mod 31)
Roots mod q: ±14 (mod 43)
Four roots beta (mod n): [100, 1061, 272, 1233]
Alice sent beta = 272
gcd(alpha + beta, n) = 31
gcd(alpha - beta, n) = 43
Result: Bob finds a non-trivial factor of n: 31
Other factor: 43
```

Slide 12 — Fairness / “coin flip” interpretation

How to phrase the “coin flip”:

- Alice’s message β determines whether Bob can factor n.
- If Alice returns a **matching root** ($\beta = \alpha$ or $\beta = n - \alpha$), Bob learns nothing → “Alice wins”.
- If Alice returns a **different root**, Bob factors n → “Bob wins”.

Probability (conceptual): 2 “safe” roots out of 4 ⇒ about 50% win chance for each, if Alice chooses randomly.

Slide 13 — Security note (important to say)

- In the demo we used small primes so we can see the math.
- In real cryptography, p and q must be **hundreds/thousands of bits**.
- Security relies on factoring $n = p \cdot q$ being computationally hard.

Slide 14 — Short conclusion

- We replaced “trust” with “math”.
- Squaring is easy, reversing it is hard without secrets.
- CRT gives Alice 4 answers; Bob’s gcd check detects the “wrong” one.

Speaker Notes (quick script)

- “Online coin toss fails because messages can be delayed/changed.”
- “Alice commits to n; Bob commits to α by sending $A=\alpha^2 \bmod n$.”
- “Knowing p and q lets Alice compute 4 roots; Bob cannot.”
- “Bob uses $\gcd(\alpha \pm \beta, n)$ to detect whether β matches his α.”
- “If gcd gives p or q, Bob wins; else Alice wins.”